Title

A Fault-Tolerant Distributed Computing Cluster
Using gRPC, Distributed Election and a Durable Task Ledger
CS324 Group Project

Abstract

This report documents a distributed computing cluster implemented in Java with gRPC. Six or more worker JVMs form an unstructured network around a neutral bootstrap node that tracks membership, mints startup priorities and persists a durable task ledger. The workers automatically elect a coordinator per term; the coordinator splits each client job into contiguous segments, distributes them to the least-loaded peer nodes, merges the partial results and streams progress to the client. The design emphasizes fault tolerance: segment workers can be killed mid-job and their work is retried, the coordinator can be killed mid-job and a newly elected successor resumes the task from the ledger, and a worker can join a running cluster and immediately start receiving segments. All three failure scenarios were exercised end-to-end and produced correct answers.

1. Introduction

The goal is a distributed system in which several worker processes cooperate to serve computational jobs under partial failure. Three job types are supported: MAX (largest of a list), PRIMECOUNT (number of primes in a list) and PRIMESUM (sum of primes in an inclusive range). The system is deliberately leader-based: a single coordinator orchestrates each job, but leadership is cheap to rotate and failover is automatic.

Communication is built on gRPC (io.grpc 1.66). The contracts live in **src/main/proto/distrilab.proto** and expose three services: BootstrapService (membership, lottery, ledger), ClusterService (election and cluster state) and ClientService (task submission and streaming updates). Every gRPC channel is cached per target host:port by **GrpcChannels** with a 30 second keep-alive.

2. System Architecture

Components (each a separate JVM, all running on localhost in the test environment):

- **Bootstrap Node** (port 1099). Tracks the worker membership list and online state via 2-second heartbeats, sequences the startup lottery, and persists every submitted task and its per-segment progress in a durable in-file ledger. Participates in neither execution nor election.
- **Worker nodes** (ports 5001 .. 5000+N). One JVM per worker. Each registers with the bootstrap, is linked into a small neighbour ring, runs the election protocol, and executes assigned segments.
- **Client**. A JavaFX GUI (ClientGUI) or a thin gRPC stub. Submits jobs and consumes server-streaming TaskUpdate messages (0%..100% progress, then TASK_DONE).

The worker holding the highest startup priority is the initial coordinator and serves at most 5 client jobs per term, after which leadership is rotated. The coordinator only assigns and merges; every segment is executed on a different peer node.

3. Election and Coordination

On cold start the bootstrap waits until all configured workers have registered, then each worker runs a randomized 150-300 ms countdown and claims a startup priority (first claim gets priority N, the last gets priority 1). The priority-N worker declares itself the initial coordinator and alerts the others.

Every worker runs a background monitor: if the cluster has no coordinator the worker initiates an election; if the current coordinator stops answering a 3-second probe the monitor demotes the local view and starts a failover election. An election message floods the neighbour ring, is processed at most once per election id, and every participant converges on the same winner. After each 5-job term the outgoing coordinator broadcasts Term_End, takes a demotion tick that raises its own JAC above the highest worker-node JAC, and a new election promotes the reachable worker with the lowest JAC (ties broken by startup priority, then worker id).

4. Task Distribution

A submitted JobRequest is tagged with a task id and recorded in the ledger. The coordinator builds contiguous segments: the segment count is min(assignable_online_workers, ceil(items / per_segment)) with per-segment 200 for PRIMESUM and 3 for MAX/PRIMECOUNT. Assignees are ranked by JAC ascending

(least-loaded first), then by startup priority descending, then worker id. The coordinator calls each assignee over gRPC with a per-segment deadline and retries an un-done segment up to 3 times on live workers, streaming each completed segment as progress toward the client.

5. Fault Tolerance

Persistence. The bootstrap ledger stores each task and every SegmentRecord, including the segment's data slice (data_start, data_length, range_start, range_end), its assigned worker, completion state and partial result. This is what makes resumption possible without re-running the whole job.

Worker kill. If a segment worker dies mid-job, its segment call eventually surfaces as an error at the coordinator, which re-assigns the segment to a live worker. The job still returns the correct total.

Coordinator kill. A killed coordinator silently drops the client's stream, so every client submit carries a 15-second deadline. On error the client resolves the new coordinator through the bootstrap (verifying each candidate with a self-confirming probe) and re-submits with the resume_task_id. The new coordinator rebuilds the remaining segments from the ledger, skipping completed ones, and finishes the task under the same task id.

Dynamic join. A late-joining worker has no startup-priority slot and therefore a negative priority, which would normally disqualify it from the lottery election. On startup it instead adopts the existing coordinator from any peer (adoptCoordinatorFromPeers), and every worker rebuilds its neighbour ring whenever the bootstrap's online membership changes (maybeResyncRingAfterMembershipChange), so the joiner becomes reachable for segment assignment.

6. Test Results

All tests ran against a single-host cluster of 6-7 workers (OpenJDK 26, io.grpc 1.66). The expected PRIMESUM answers were independently verified: 1..200000 = 1709600813, 1..1,000,000 = 37550402023, 1..2,000,000 = 142913828922, 1..8,000,000 = 2080483502248.

Scenario	Expected result	Verdict
Worker kill (1..200000)	1709600813	PASS
Worker kill (1..1,000,000)	37550402023	PASS
Coordinator kill (1..2,000,000)	142913828922	PASS
Coordinator kill (1..8,000,000)	2080483502248	PASS
Dynamic join (1..4,000,000)	544501644261	PASS

In the worker-kill runs the segments executing on the killed node were re-assigned to live workers and the client stream never broke. In the coordinator-kill runs the client stream was broken (read failed with UNAVAILABLE), the new coordinator was resolved through the bootstrap and the task re-submitted under the original resume_task_id; the client trace showed the full lifecycle: updates at 0% then 33%, the UNAVAILABLE error, resolution, resumption, and correct completion. The dynamic-join run showed worker 7 logging **adopted existing coordinator worker 3 from peer 1** and subsequently computing the range [3333335, 4000000].

7. Limitations and Future Work

The current implementation assumes a single host for demo purposes; deployment across real networks would rely on the same gRPC contracts with per-host addresses. The eventual-consistency window during a coordinator failover means a client may briefly re-attach to an old view (mitigated by the deadline and the self-confirming resolve loop). Possible extensions include checkpointed aggregate state on workers, per-segment checksum verification, and coordinator-aware client-side service discovery.

8. Appendix: Documented Run Evidence

All three fault-tolerance scenarios were exercised end-to-end with the smoke-test clients named WorkerKillClient and CoordinatorKillClient (plus AsyncSubmit as a healthy-cluster control). The complete commit history of the project is captured in git-history.txt, and operating instructions in README.md.